



# Rapport Technique de l'API de Gestion de Bases de Données

## Programmation Orientée Objet en Java

Filière Génie Informatique 1

**Étudiante :**  
TAFRAOUTI Sanae

**Encadré par :** Prof.ELBOUHDIDI Jaber

**Année Universitaire :** 2024-2025  
**Semestre :** 2

15 mai 2025



# Table des matières

|          |                                          |           |
|----------|------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                      | <b>3</b>  |
| 1.1      | Contexte . . . . .                       | 3         |
| 1.2      | Objectifs . . . . .                      | 3         |
| <b>2</b> | <b>Architecture du Projet</b>            | <b>4</b>  |
| 2.1      | Structure du Projet . . . . .            | 4         |
| 2.2      | Dépendances . . . . .                    | 5         |
| <b>3</b> | <b>Interface Principale</b>              | <b>6</b>  |
| 3.1      | DatabaseManager . . . . .                | 6         |
| <b>4</b> | <b>Implémentation</b>                    | <b>7</b>  |
| 4.1      | MySQLManager . . . . .                   | 7         |
| 4.2      | PostgreSQLManager . . . . .              | 9         |
| 4.3      | SQLServerManager . . . . .               | 11        |
| 4.4      | DBConfigLoader . . . . .                 | 13        |
| 4.5      | DatabaseManagerTest . . . . .            | 14        |
| 4.6      | Classe Main . . . . .                    | 15        |
| <b>5</b> | <b>Fonctionnalités</b>                   | <b>17</b> |
| 5.1      | Opérations Supportées . . . . .          | 17        |
| 5.2      | Exemple d'Utilisation . . . . .          | 17        |
| <b>6</b> | <b>Configuration</b>                     | <b>19</b> |
| 6.1      | Fichier de Configuration . . . . .       | 19        |
| <b>7</b> | <b>Test</b>                              | <b>20</b> |
| 7.1      | Run : DatabaseManagerTest.java . . . . . | 20        |
| 7.2      | Run : mvn test . . . . .                 | 21        |
| 7.3      | Run : Main.java . . . . .                | 22        |
| <b>8</b> | <b>Conclusion</b>                        | <b>26</b> |

# Chapitre 1

## Introduction

### 1.1 Contexte

Ce rapport présente une analyse détaillée de l'API de gestion de bases de données développée dans le cadre du cours de Programmation Orientée Objet en Java. Cette API permet une interaction unifiée avec différentes bases de données (MySQL, PostgreSQL, SQL Server) tout en maintenant une interface cohérente et extensible.

### 1.2 Objectifs

- Développer une API unifiée pour la gestion de différentes bases de données
- Implémenter les opérations CRUD (Create, Read, Update, Delete)
- Assurer la portabilité et l'extensibilité du code
- Maintenir une architecture propre et modulaire

# Chapitre 2

## Architecture du Projet

### 2.1 Structure du Projet

Le projet suit une architecture Maven standard avec la structure suivante :

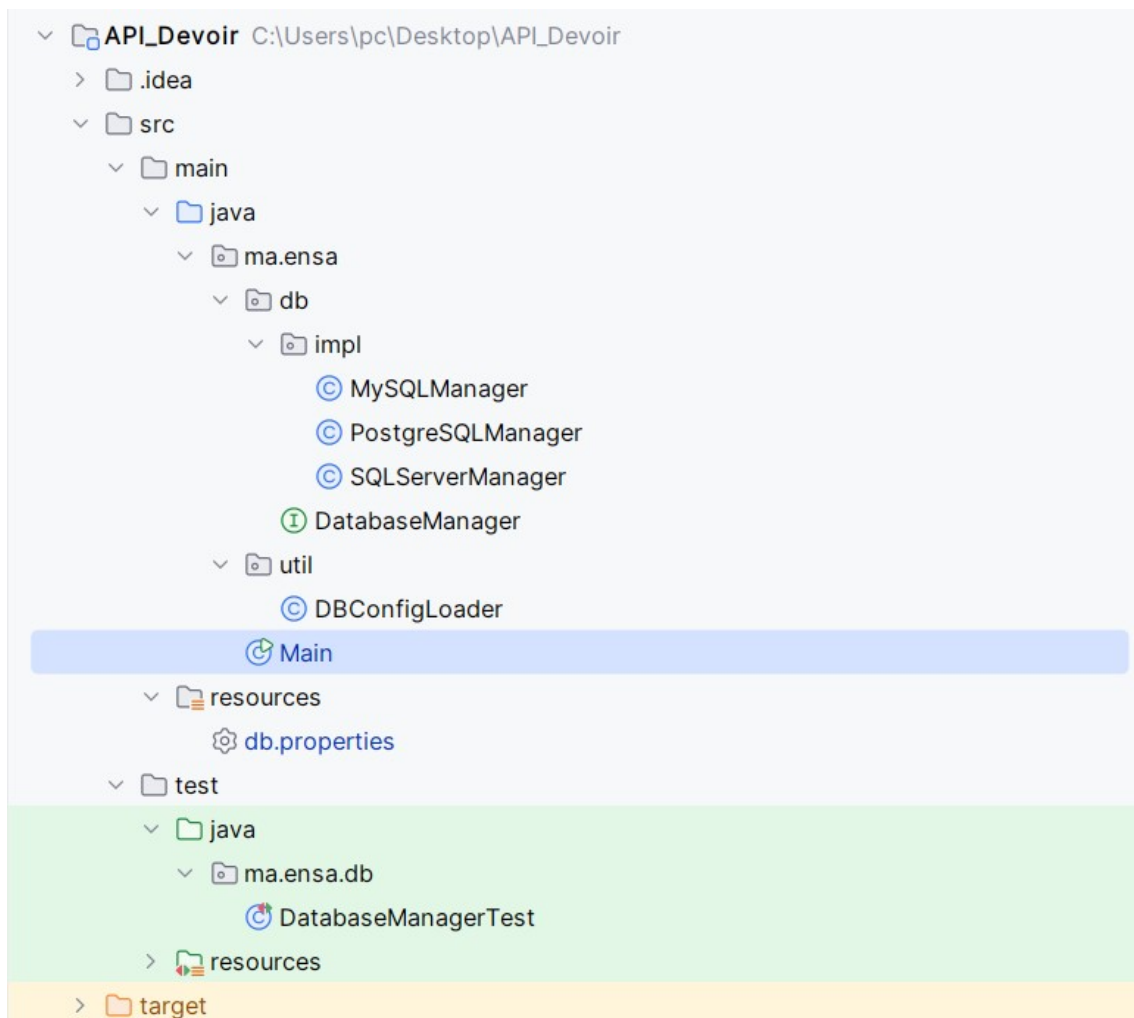


FIGURE 2.1 – Structure du Projet

## 2.2 Dépendances

Le projet utilise les dépendances suivantes (définies dans pom.xml) :

- **Lombok (1.18.38)**
  - Réduction du code boilerplate
  - Génération automatique des getters/setters
  - Support des annotations @Data, @Builder, etc.
- **Connecteurs de base de données**
  - MySQL Connector (9.0.0)
    - Support complet de MySQL 8.0+
    - Gestion des transactions
    - Support des requêtes préparées
  - PostgreSQL (42.7.5)
    - Support des types de données avancés
    - Gestion des connexions SSL
  - Microsoft SQL Server (12.10.0)
    - Support de l'authentification Windows
    - Gestion des procédures stockées
- **JUnit (5.11.4)**
  - Tests unitaires
  - Assertions avancées
  - Support des tests paramétrés

# Chapitre 3

## Interface Principale

### 3.1 DatabaseManager

L'interface principale qui définit les opérations communes pour toutes les implémentations :

```
1 public interface DatabaseManager {
2     /* Etablit une connexion avec la base de donnees
3     * @throws Exception si la connexion choue
4     */
5     void connect() throws Exception;
6
7     /* Ferme la connexion avec la base de donnees
8     * @throws Exception si la deconnexion echoue
9     */
10    void disconnect() throws Exception;
11
12    /* Execute une requete SELECT
13    * @param query La requete SQL a executer
14    * @return Liste des resultats sous forme de Map
15    * @throws Exception si l'execution echoue
16    */
17    List<Map<String, Object>> executeQuery(String query)
18        throws Exception;
19
20    /*Execute une requete INSERT/UPDATE/DELETE
21    * @param query La requete SQL a executer
22    * @return Nombre de lignes affectees
23    * @throws Exception si l'ex cution echoue
24    */
25    int executeUpdate(String query) throws Exception;
26
27    /*Verifie si la connexion est active
28    * @return true si connecte, false sinon
29    */
30    boolean isConnected();
31 }
```

# Chapitre 4

## Implémentation

### 4.1 MySQLManager

Implémentation du gestionnaire pour MySQL :

```
1 package ma.ensa.db.impl;
2
3 import lombok.Getter;
4 import ma.ensa.db.DatabaseManager;
5
6 import java.sql.*;
7 import java.util.ArrayList;
8 import java.util.HashMap;
9 import java.util.List;
10 import java.util.Map;
11
12 @Getter
13 public class MySQLManager implements DatabaseManager {
14     private Connection connection;
15     private String url;
16     private String username;
17     private String password;
18
19     public MySQLManager(String url, String username, String
20         password) {
21         this.url = "jdbc:mysql://" + url;
22         this.username = username;
23         this.password = password;
24     }
25
26     @Override
27     public void connect() throws Exception {
28         try {
29             Class.forName("com.mysql.cj.jdbc.Driver");
30             this.connection = DriverManager.getConnection(url
31                 , username, password);
32         } catch (ClassNotFoundException | SQLException e) {
33             throw new Exception("MySQL Connection Error: " +
34                 e.getMessage());
35         }
36     }
37 }
```



```

32     }
33 }
34
35 @Override
36 public void disconnect() throws Exception {
37     try {
38         if (connection != null && !connection.isClosed())
39             {
40                 connection.close();
41             }
42     } catch (SQLException e) {
43         throw new Exception("MySQL Disconnection Error: "
44             + e.getMessage());
45     }
46 }
47
48 @Override
49 public List<Map<String, Object>> executeQuery(String
50     query) throws Exception {
51     List<Map<String, Object>> resultList = new ArrayList
52         <>();
53
54     try (Statement stmt = connection.createStatement();
55         ResultSet rs = stmt.executeQuery(query)) {
56
57         ResultSetMetaData metaData = rs.getMetaData();
58         int columnCount = metaData.getColumnCount();
59
60         while (rs.next()) {
61             Map<String, Object> row = new HashMap<>();
62             for (int i = 1; i <= columnCount; i++) {
63                 row.put(metaData.getColumnName(i), rs.
64                     getObject(i));
65             }
66             resultList.add(row);
67         }
68     } catch (SQLException e) {
69         throw new Exception("MySQL Query Execution Error:
70             " + e.getMessage());
71     }
72
73     return resultList;
74 }
75
76 @Override
77 public int executeUpdate(String query) throws Exception {
78     try (Statement stmt = connection.createStatement()) {
79         return stmt.executeUpdate(query);
80     } catch (SQLException e) {
81         throw new Exception("MySQL Update Execution Error
82             : " + e.getMessage());
83     }
84 }

```

```

79     @Override
80     public boolean isConnected() {
81         try {
82             return connection != null && !connection.isClosed
83                 ();
84         } catch (SQLException e) {
85             return false;
86         }
87     }

```

## 4.2 PostgreSQLManager

Implémentation du gestionnaire pour PostgreSQL :

```

1  package ma.ensa.db.impl;
2
3  import lombok.Getter;
4  import ma.ensa.db.DatabaseManager;
5
6  import java.sql.*;
7  import java.util.ArrayList;
8  import java.util.HashMap;
9  import java.util.List;
10 import java.util.Map;
11
12 @Getter
13 public class PostgreSQLManager implements DatabaseManager {
14     private Connection connection;
15     private String url;
16     private String username;
17     private String password;
18
19     public PostgreSQLManager(String url, String username,
20         String password) {
21         this.url = "jdbc:postgresql://" + url;
22         this.username = username;
23         this.password = password;
24     }
25
26     @Override
27     public void connect() throws Exception {
28         try {
29             Class.forName("org.postgresql.Driver");
30             this.connection = DriverManager.getConnection(url
31                 , username, password);
32         } catch (ClassNotFoundException e) {
33             throw new Exception("PostgreSQL Driver not found:
34                 " + e.getMessage());
35         } catch (SQLException e) {
36             throw new Exception("PostgreSQL Connection Error:
37                 " + e.getMessage());
38         }
39     }

```

```

34     }
35 }
36
37 @Override
38 public void disconnect() throws Exception {
39     try {
40         if (connection != null && !connection.isClosed())
41             {
42                 connection.close();
43             }
44     } catch (SQLException e) {
45         throw new Exception("PostgreSQL Disconnection
46             Error: " + e.getMessage());
47     }
48 }
49
50 @Override
51 public List<Map<String, Object>> executeQuery(String
52     query) throws Exception {
53     List<Map<String, Object>> resultList = new ArrayList
54         <>();
55
56     try (Statement stmt = connection.createStatement();
57         ResultSet rs = stmt.executeQuery(query)) {
58
59         ResultSetMetaData metaData = rs.getMetaData();
60         int columnCount = metaData.getColumnCount();
61
62         while (rs.next()) {
63             Map<String, Object> row = new HashMap<>();
64             for (int i = 1; i <= columnCount; i++) {
65                 row.put(metaData.getColumnName(i), rs.
66                     getObject(i));
67             }
68             resultList.add(row);
69         }
70     } catch (SQLException e) {
71         throw new Exception("PostgreSQL Query Execution
72             Error: " + e.getMessage());
73     }
74
75     return resultList;
76 }
77
78 @Override
79 public int executeUpdate(String query) throws Exception {
80     try (Statement stmt = connection.createStatement()) {
81         return stmt.executeUpdate(query);
82     } catch (SQLException e) {
83         throw new Exception("PostgreSQL Update Execution
84             Error: " + e.getMessage());
85     }
86 }

```

```

81  @Override
82  public boolean isConnected() {
83      try {
84          return connection != null && !connection.isClosed
85              ();
86      } catch (SQLException e) {
87          return false;
88      }
89  }
90
91  public List<String> getDatabaseList() throws Exception {
92      List<String> databases = new ArrayList<>();
93      try (Statement stmt = connection.createStatement();
94           ResultSet rs = stmt.executeQuery("SELECT datname
95                                           FROM pg_database WHERE datistemplate = false
96                                           ")) {
97
98          while (rs.next()) {
99              databases.add(rs.getString("datname"));
100          }
101      } catch (SQLException e) {
102          throw new Exception("Error fetching PostgreSQL
103                              databases: " + e.getMessage());
104      }
105      return databases;
106  }
107 }

```

## 4.3 SQLServerManager

Implémentation du gestionnaire pour SQL Server :

```

1  package ma.ensa.db.impl;
2
3  import lombok.Getter;
4  import ma.ensa.db.DatabaseManager;
5
6  import java.sql.*;
7  import java.util.ArrayList;
8  import java.util.HashMap;
9  import java.util.List;
10 import java.util.Map;
11
12 @Getter
13 public class SQLServerManager implements DatabaseManager {
14     private Connection connection;
15     private String url;
16     private String username;
17     private String password;
18
19     public SQLServerManager(String url, String username,
20                             String password) {

```

```

20         this.url = "jdbc:sqlserver://" + url + ";databaseName
           =TestAPI;integratedSecurity=true;encrypt=false;
           trustServerCertificate=true;";
21         this.username = username;
22         this.password = password;
23
24         System.setProperty("java.library.path", "C:/sqljdbc_<
           version>/enu/auth/x64");
25     }
26
27     @Override
28     public void connect() throws Exception {
29         try {
30             Class.forName("com.microsoft.sqlserver.jdbc.
           SQLServerDriver");
31             this.connection = DriverManager.getConnection(url
           );
32         } catch (ClassNotFoundException e) {
33             throw new Exception("SQL Server Driver not found:
           " + e.getMessage());
34         } catch (SQLException e) {
35             throw new Exception("SQL Server Connection Error:
           " + e.getMessage());
36         }
37     }
38
39     @Override
40     public void disconnect() throws Exception {
41         try {
42             if (connection != null && !connection.isClosed())
43             {
44                 connection.close();
45             }
46         } catch (SQLException e) {
47             throw new Exception("SQL Server Disconnection
           Error: " + e.getMessage());
48         }
49     }
50
51     @Override
52     public List<Map<String, Object>> executeQuery(String
           query) throws Exception {
53         List<Map<String, Object>> resultList = new ArrayList
           <>();
54
55         try (Statement stmt = connection.createStatement();
           ResultSet rs = stmt.executeQuery(query)) {
56
57             ResultSetMetaData metaData = rs.getMetaData();
58             int columnCount = metaData.getColumnCount();
59
60             while (rs.next()) {
61                 Map<String, Object> row = new HashMap<>();
62                 for (int i = 1; i <= columnCount; i++) {

```

```

63         row.put(metaData.getColumnName(i), rs.
64             getObject(i));
65     }
66     resultList.add(row);
67 } catch (SQLException e) {
68     throw new Exception("SQL Server Query Execution
69         Error: " + e.getMessage());
70 }
71
72     return resultList;
73 }
74
75 @Override
76 public int executeUpdate(String query) throws Exception {
77     try (Statement stmt = connection.createStatement()) {
78         return stmt.executeUpdate(query);
79     } catch (SQLException e) {
80         throw new Exception("SQL Server Update Execution
81             Error: " + e.getMessage());
82     }
83 }
84
85 @Override
86 public boolean isConnected() {
87     try {
88         return connection != null && !connection.isClosed
89             ();
90     } catch (SQLException e) {
91         return false;
92     }
93 }

```

## 4.4 DBConfigLoader

Classe utilitaire pour le chargement de la configuration :

```

1 package ma.ensa.util;
2
3 import java.io.FileInputStream;
4 import java.io.IOException;
5 import java.util.Properties;
6
7 public class DBConfigLoader {
8     /**
9      * Charge les propri  t  s depuis un fichier
10     * @param filename Nom du fichier de configuration
11     * @return Properties charg  es
12     * @throws IOException si le fichier ne peut pas   tre lu
13     */
14     public static Properties loadProperties(String filename)
15         throws IOException {

```

```

15     Properties props = new Properties();
16     try (FileInputStream fis = new FileInputStream(
17         filename)) {
18         props.load(fis);
19     }
20     return props;
21 }

```

## 4.5 DatabaseManagerTest

```

1 package ma.ensa.db;
2
3 import ma.ensa.db.impl.MySQLManager;
4 import ma.ensa.util.DBConfigLoader;
5 import org.junit.jupiter.api.*;
6
7 import java.util.List;
8 import java.util.Map;
9 import java.util.Properties;
10
11 import static org.junit.jupiter.api.Assertions.*;
12
13 @TestMethodOrder(MethodOrderer.OrderAnnotation.class)
14 public class DatabaseManagerTest {
15     private static DatabaseManager dbManager;
16
17     @BeforeAll
18     static void setup() throws Exception {
19         Properties props = DBConfigLoader.loadProperties("db.
20             properties");
21         dbManager = new MySQLManager(
22             props.getProperty("mysql.url") + "/" + props.
23                 getProperty("mysql.database"),
24             props.getProperty("mysql.username"),
25             props.getProperty("mysql.password")
26         );
27         dbManager.connect();
28     }
29
30     @AfterAll
31     static void cleanup() throws Exception {
32         if (dbManager != null && dbManager.isConnected()) {
33             dbManager.disconnect();
34         }
35     }
36
37     @Test
38     @Order(1)
39     void testConnection() {
40         assertTrue(dbManager.isConnected());
41     }
42 }

```

```

40
41 @Test
42 @Order(2)
43 void testQueryExecution() throws Exception {
44     List<Map<String, Object>> results = dbManager.
45         executeQuery("SELECT * FROM employees");
46     assertFalse(results.isEmpty());
47     assertEquals(3, results.size());
48 }
49
50 @Test
51 @Order(3)
52 void testInsertAndDelete() throws Exception {
53     // Insertion
54     int inserted = dbManager.executeUpdate(
55         "INSERT INTO employees VALUES (4, 'Test', '
56         User', 'test@email.com', 3000.00)");
57     assertEquals(1, inserted);
58
59     // Verification
60     List<Map<String, Object>> results = dbManager.
61         executeQuery(
62             "SELECT * FROM employees WHERE id = 4");
63     assertEquals(1, results.size());
64     assertEquals("Test", results.get(0).get("first_name")
65         );
66
67     // Nettoyage
68     int deleted = dbManager.executeUpdate("DELETE FROM
69         employees WHERE id = 4");
70     assertEquals(1, deleted);
71 }
72
73 @Test
74 @Order(4)
75 void testInvalidQuery() {
76     Exception exception = assertThrows(Exception.class,
77         () -> {
78             dbManager.executeQuery("SELECT * FROM
79                 non_existent_table");
80         });
81     assertTrue(exception.getMessage().contains("Error"));
82 }
83 }

```

## 4.6 Classe Main

Point d'entrée de l'application qui démontre l'utilisation de l'API :

```

1 package ma.ensa;
2
3 import ma.ensa.db.DatabaseManager;

```



```

4 import ma.ensa.db.impl.MySQLManager;
5 import ma.ensa.db.impl.PostgreSQLManager;
6 import ma.ensa.db.impl.SQLServerManager;
7 import ma.ensa.util.DBConfigLoader;
8
9 import java.util.List;
10 import java.util.Map;
11 import java.util.Properties;
12
13 public class Main {
14     public static void main(String[] args) {
15         try {
16             Properties props = DBConfigLoader.loadProperties(
17                 "db.properties");
18
19             // Test MySQL
20             System.out.println("\n=== MySQL (XAMPP) ===");
21             DatabaseManager mysqlManager = new MySQLManager(
22                 props.getProperty("mysql.url") + "/" +
23                 props.getProperty("mysql.database"),
24                 props.getProperty("mysql.username"),
25                 props.getProperty("mysql.password")
26             );
27             testDatabase(mysqlManager, "employees");
28
29             // Test PostgreSQL
30             System.out.println("\n=== PostgreSQL (pgAdmin) ===");
31             DatabaseManager postgresManager = new
32                 PostgreSQLManager(
33                     props.getProperty("postgresql.url") + "/"
34                     + props.getProperty("postgresql.
35                         database"),
36                     props.getProperty("postgresql.username"),
37                     props.getProperty("postgresql.password")
38                 );
39             testDatabase(postgresManager, "employees");
40
41             // Test SQL Server
42             System.out.println("\n=== SQL Server ===");
43             DatabaseManager sqlServerManager = new
44                 SQLServerManager(
45                     props.getProperty("sqlserver.url"),
46                     props.getProperty("sqlserver.username"),
47                     props.getProperty("sqlserver.password")
48                 );
49             testDatabase(sqlServerManager, "employees");
50
51         } catch (Exception e) {
52             e.printStackTrace();
53         }
54     }
55 }

```

# Chapitre 5

## Fonctionnalités

### 5.1 Opérations Supportées

- **Connexion à la base de données**
  - Support de différents types d'authentification
  - Gestion des timeouts
  - Vérification de la validité de la connexion
- **Exécution de requêtes SELECT**
  - Récupération de données
  - Support des jointures
  - Gestion des résultats null
- **Exécution de requêtes INSERT/UPDATE/DELETE**
  - Insertion de nouvelles données
  - Mise à jour des données existantes
  - Suppression de données
- **Gestion des connexions**
  - Ouverture de connexion
  - Fermeture propre
  - Vérification de l'état

### 5.2 Exemple d'Utilisation

```
1 private static void testDatabase(DatabaseManager manager,
2     String tableName)
3     throws Exception {
4     try {
5         // Connexion
6         manager.connect();
7         System.out.println("Statut de la connexion : " +
8             manager.isConnected());
9
10        // Execution d'une requete SELECT
11        System.out.println("\n--- Avant l'insertion et
12            suppression ---");
```

```

10      List<Map<String, Object>> results1 = manager.
        executeQuery
11          ("SELECT * FROM " + tableName);
12
13      // Affichage des resultats
14      System.out.println("\nResultats de la requete :");
15      for (Map<String, Object> row : results1) {
16          System.out.println(row);
17      }
18
19      // Exemple d'insertion
20      int affectedRows1 = manager.executeUpdate(
21          "INSERT INTO " + tableName + " VALUES (4, 'Test1
            ', 'User1', 'test1.user1@email.com', 4700.00)");
22      System.out.println("Lignes ins r es (4, 'Test1', '
            User1',
23          'test1.user1@email.com', 4700.00) : " +
            affectedRows1);
24
25      int affectedRows2 = manager.executeUpdate(
26          "INSERT INTO " + tableName + " VALUES (5, 'Test2
            ', 'User2', 'test2.user2@email.com', 3000.00)");
27      System.out.println("Lignes ins r es (5, 'Test2', '
            User2',
28          'test2.user2@email.com', 3000.00) : " +
            affectedRows2);
29
30      // Nettoyage
31      manager.executeUpdate("DELETE FROM " + tableName + "
            WHERE id = 4");
32      System.out.println("\nSuppression de l'enregistrement
            : (4,
33          'Test1', 'User1', 'test1.user1@email.com',
            4700.00) ");
34
35      System.out.println("\n\n--- Apres l'insertion et
            suppression ---");
36      List<Map<String, Object>> results2 = manager.
        executeQuery("SELECT * FROM " + tableName);
37
38      System.out.println("Nombre d'enregistrements : " +
            results2.size());
39      for (Map<String, Object> row : results2) {
40          System.out.println(row);
41      }
42
43      } finally {
44          // Fermeture propre
45          manager.disconnect();
46          System.out.println("\nConnexion fermee");
47      }
48 }

```

# Chapitre 6

## Configuration

### 6.1 Fichier de Configuration

Exemple de fichier db.properties avec des commentaires détaillés :

```
1 // Configuration MySQL
2 // URL de connexion avec le port par d faut
3 mysql.url=jdbc:mysql:localhost:3306
4 // Nom de la base de donn es
5 mysql.database=test_api
6 // Identifiants de connexion
7 mysql.username=root
8 mysql.password=
9
10 // Configuration PostgreSQL
11 // URL de connexion avec le port par d faut
12 postgresql.url=localhost:5432
13 // Nom de la base de donn es
14 postgresql.database=test_api
15 // Identifiants de connexion
16 postgresql.username=postgres
17 postgresql.password=passe
18
19 // Configuration SQL Server
20 // URL de connexion avec le port par d faut et le nom de la
    base
21 sqlserver.url=localhost:1433
22 sqlserver.database=TestAPI
23 // Identifiants de connexion
24 sqlserver.username=TAFRAOUTI-SANAE\pc
25 sqlserver.password=
```

# Chapitre 7

## Test

### 7.1 Run : DatabaseManagerTest.java

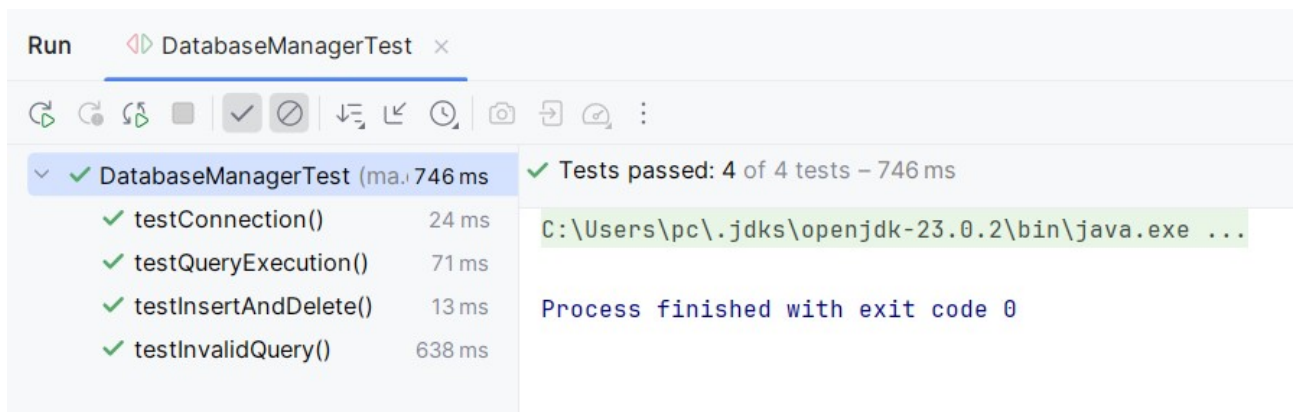
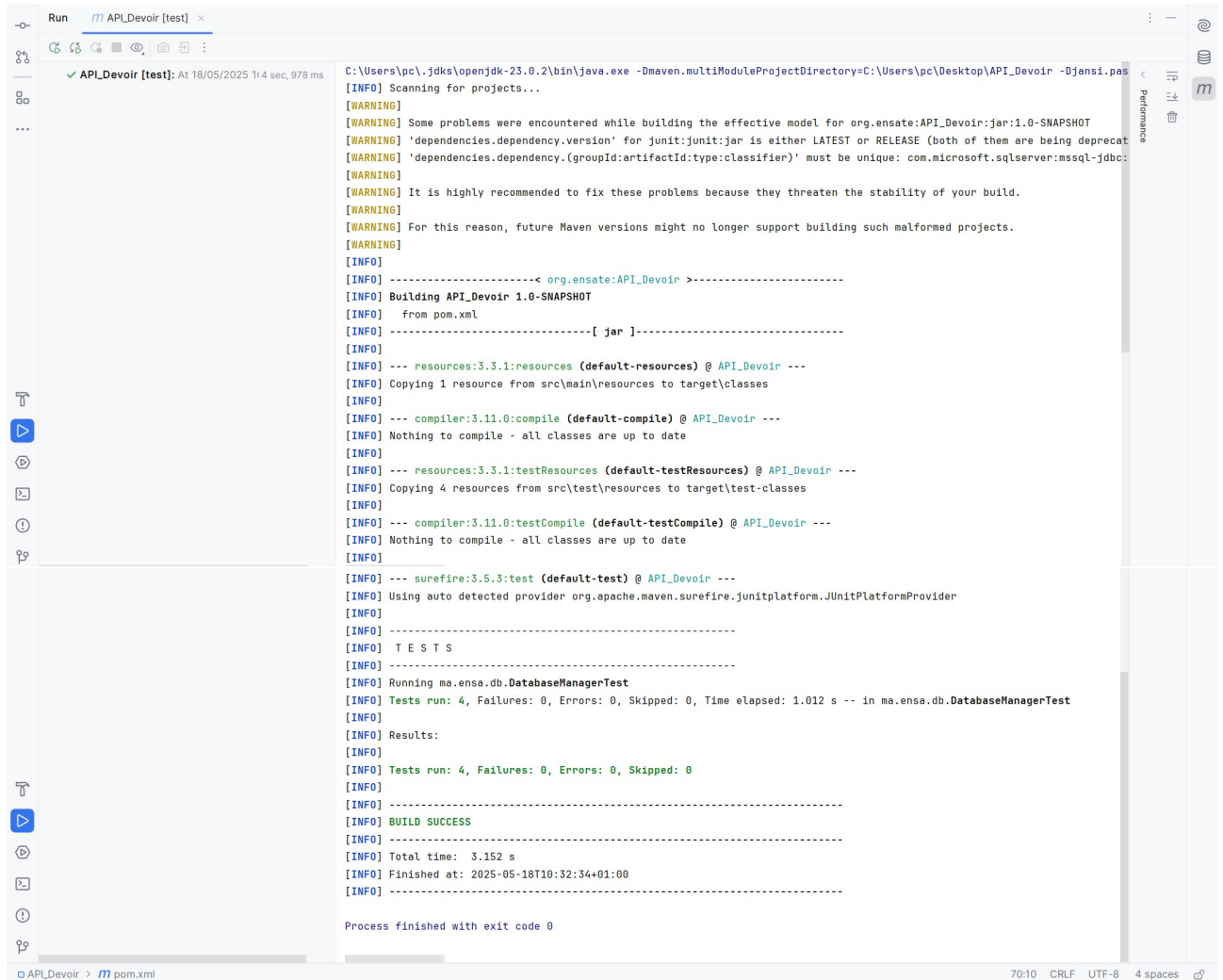


FIGURE 7.1 – Exécution DatabaseManagerTest.java

## 7.2 Run : mvn test

Exécution la commande :

```
1 mvn test
```



```
Run APL_Devoir [test] x
✓ APL_Devoir [test]: At 18/05/2025 11:4 sec, 978 ms
C:\Users\pc\.jdk\openjdk-23.0.2\bin\java.exe -Dmaven.multiModuleProjectDirectory=C:\Users\pc\Desktop\API_Devoir -Djansi.pas
[INFO] Scanning for projects...
[WARNING]
[WARNING] Some problems were encountered while building the effective model for org.ensate:API_Devoir:jar:1.0-SNAPSHOT
[WARNING] 'dependencies.dependency.version' for junit:junit:jar is either LATEST or RELEASE (both of them are being deprecate
[WARNING] 'dependencies.dependency.(groupId:artifactId:type:classifier)' must be unique: com.microsoft.sqlserver:mssql-jdbc:
[WARNING]
[WARNING] It is highly recommended to fix these problems because they threaten the stability of your build.
[WARNING]
[WARNING] For this reason, future Maven versions might no longer support building such malformed projects.
[WARNING]
[INFO]
[INFO] -----< org.ensate:API_Devoir >-----
[INFO] Building API_Devoir 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ API_Devoir ---
[INFO] Copying 1 resource from src\main\resources to target\classes
[INFO]
[INFO] --- compiler:3.11.0:compile (default-compile) @ API_Devoir ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ API_Devoir ---
[INFO] Copying 4 resources from src\test\resources to target\test-classes
[INFO]
[INFO] --- compiler:3.11.0:testCompile (default-testCompile) @ API_Devoir ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- surefire:3.5.3:test (default-test) @ API_Devoir ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running ma.ensa.db.DatabaseManagerTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.012 s -- in ma.ensa.db.DatabaseManagerTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 3.152 s
[INFO] Finished at: 2025-05-18T10:32:34+01:00
[INFO] -----
Process finished with exit code 0
```

FIGURE 7.2 – Exécution la commande : mvn test

## 7.3 Run : Main.java

### Test MySQL (XAMPP)

```
Run Main x
C:\Users\pc\.jdk\openjdk-23.0.2\bin\java.exe ...

=== MySQL (XAMPP) ===
Connecté : true

--- Avant l'insertion et suppression ---
Nombre d'enregistrements : 3
{last_name=Tafraouti, id=1, salary=3500.00, first_name=Sanae, email=sanae.tafraouti@gmail.com}
{last_name=Hamani, id=2, salary=4200.50, first_name=Yaakoub, email=yaakoub.hamani@gmail.com}
{last_name=Akrami, id=3, salary=3800.75, first_name=Mohamed, email=mohamed.akrami@gmail.com}
Lignes insérées (4, 'Test1', 'User1', 'test1.user1@email.com', 4700.00) : 1
Lignes insérées (5, 'Test2', 'User2', 'test2.user2@email.com', 3000.00) : 1

Suppression de l'enregistrement : (4, 'Test1', 'User1', 'test1.user1@email.com', 4700.00)

--- Après l'insertion et suppression ---
Nombre d'enregistrements : 4
{last_name=Tafraouti, id=1, salary=3500.00, first_name=Sanae, email=sanae.tafraouti@gmail.com}
{last_name=Hamani, id=2, salary=4200.50, first_name=Yaakoub, email=yaakoub.hamani@gmail.com}
{last_name=Akrami, id=3, salary=3800.75, first_name=Mohamed, email=mohamed.akrami@gmail.com}
{last_name=User2, id=5, salary=3000.00, first_name=Test2, email=test2.user2@email.com}
Déconnecté
```

FIGURE 7.3 – Test MySQL

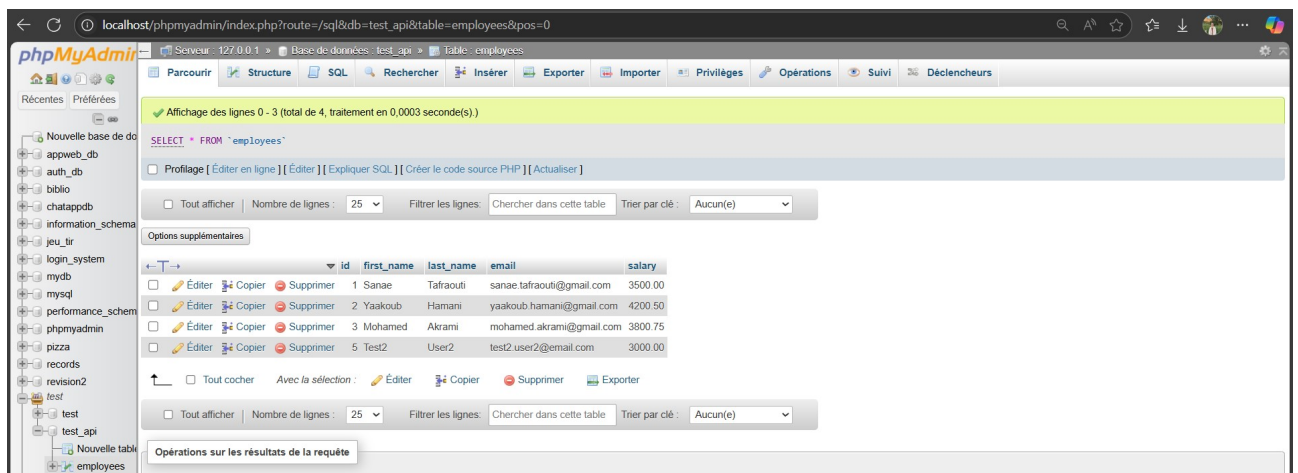


FIGURE 7.4 – Table **employees** dans phpMyAdmin

## Test PostgreSQL (pgAdmin)

```
Run Main x
=== PostgreSQL (pgAdmin) ===
Connecté : true

--- Avant l'insertion et suppression ---
Nombre d'enregistrements : 3
{last_name=Tafraouti, id=1, salary=3500.00, first_name=Sanae, email=sanae.tafrouti@gmail.com}
{last_name=Hamani, id=2, salary=4200.50, first_name=Yaakoub, email=yaakoub.hamani@gmail.com}
{last_name=Akrami, id=3, salary=3800.75, first_name=Mohamed, email=mohamed.akrami@gmail.com}
Lignes insérées (4, 'Test1', 'User1', 'test1.user1@email.com', 4700.00) : 1
Lignes insérées (5, 'Test2', 'User2', 'test2.user2@email.com', 3000.00) : 1

Suppression de l'enregistrement : (4, 'Test1', 'User1', 'test1.user1@email.com', 4700.00)

--- Après l'insertion et suppression ---
Nombre d'enregistrements : 4
{last_name=Tafraouti, id=1, salary=3500.00, first_name=Sanae, email=sanae.tafrouti@gmail.com}
{last_name=Hamani, id=2, salary=4200.50, first_name=Yaakoub, email=yaakoub.hamani@gmail.com}
{last_name=Akrami, id=3, salary=3800.75, first_name=Mohamed, email=mohamed.akrami@gmail.com}
{last_name=User2, id=5, salary=3000.00, first_name=Test2, email=test2.user2@email.com}
Déconnecté
```

FIGURE 7.5 – Test PostgreSQL

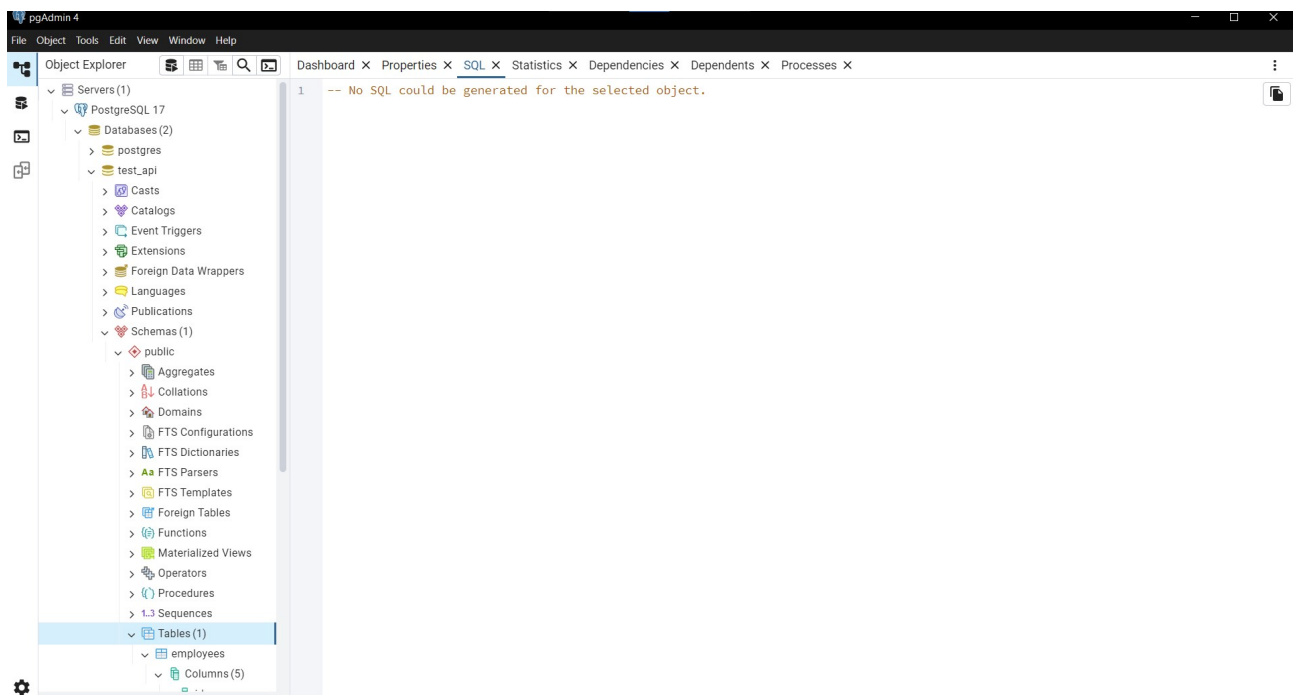
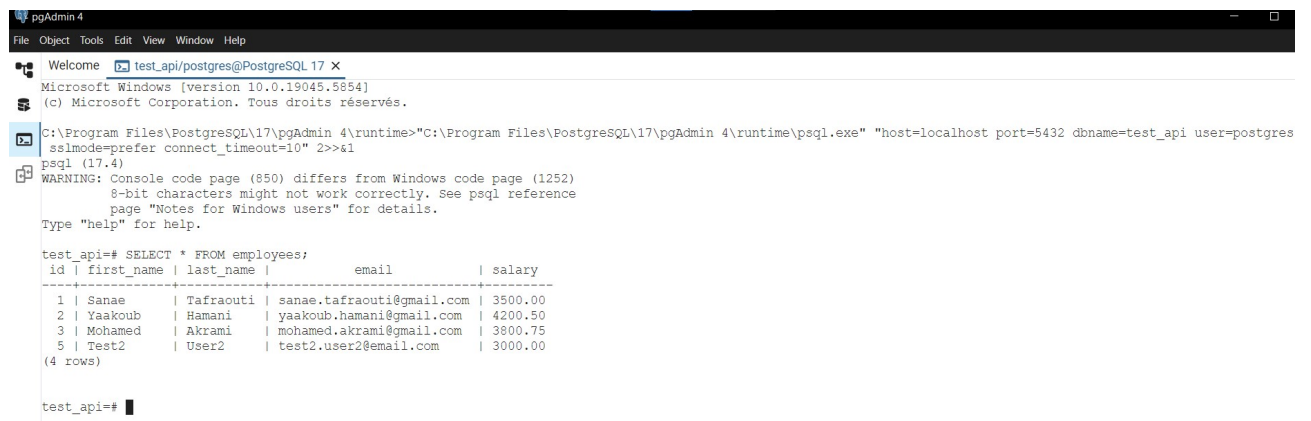


FIGURE 7.6 – pgAdmin 4





The screenshot shows the pgAdmin 4 application window. The title bar reads "pgAdmin 4". The menu bar includes "File", "Object", "Tools", "Edit", "View", "Window", and "Help". The main window displays a terminal-like interface with the following content:

```
Welcome test_api/postgres@PostgreSQL 17 X
Microsoft Windows [version 10.0.19045.5854]
(c) Microsoft Corporation. Tous droits réservés.

C:\Program Files\PostgreSQL\17\pgAdmin 4\runtime>"C:\Program Files\PostgreSQL\17\pgAdmin 4\runtime\psql.exe" "host=localhost port=5432 dbname=test_api user=postgres
sslmode=prefer connect_timeout=10" 2>>41
psql (17.4)
WARNING: Console code page (850) differs from Windows code page (1252)
8-bit characters might not work correctly. See psql reference
page "Notes for Windows users" for details.
Type "help" for help.

test_api=# SELECT * FROM employees;
 id | first_name | last_name | email | salary
-----+-----+-----+-----+-----
  1 | Sanae     | Taфраouti | sanae.taфраouti@gmail.com | 3500.00
  2 | Yaakoub   | Hamani    | yaakoub.hamani@gmail.com  | 4200.50
  3 | Mohamed   | Akrami     | mohamed.akrami@gmail.com  | 3800.75
  5 | Test2     | User2      | test2.user2@email.com     | 3000.00
(4 rows)

test_api=#
```

FIGURE 7.7 – Table **employees** dans pgAdmin 4

## Test SQL Server

```
=== SQL Server ===
Déconnecté

java.lang.Exception Create breakpoint : SQL Server Connection Error: Ce pilote n'est pas configuré pour l'authentification intégrée. ClientConnectionId:137cba90-1567-4a8c-b4b9-
    at ma.ensa.db.impl.SQLServerManager.connect(SQLServerManager.java:35)
    at ma.ensa.Main.testDatabase(Main.java:52)
    at ma.ensa.Main.main(Main.java:43)

Process finished with exit code 0
```

FIGURE 7.8 – Test SQL Server

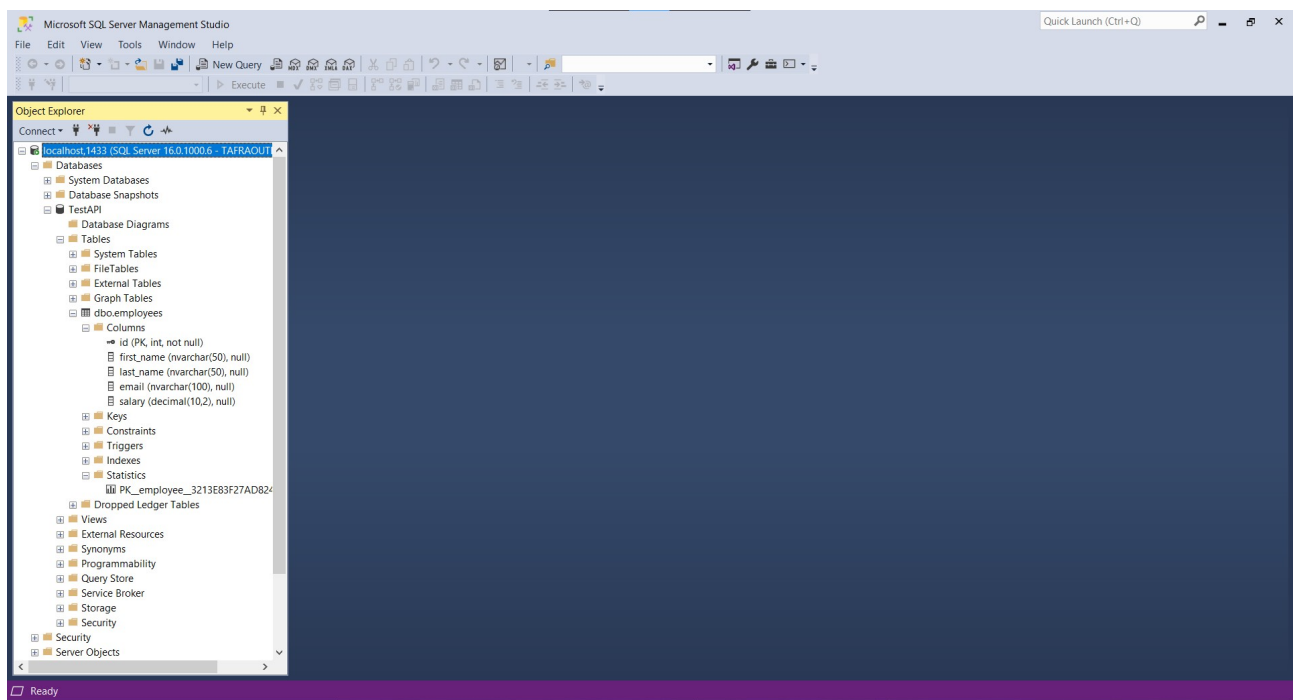


FIGURE 7.9 – SQL Server Management Studio (SSMS)

# Chapitre 8

## Conclusion

Cette API fournit une solution robuste et flexible pour l'interaction avec différentes bases de données. Son architecture modulaire permet une extension facile pour supporter d'autres types de bases de données tout en maintenant une interface cohérente. Les bonnes pratiques de programmation ont été suivies pour assurer la maintenabilité et l'évolutivité du code.